

Modeling Expected Goals in NHL Hockey Using Public Play-by-Play Data

Tyler Forgone

May 1, 2026

Abstract

Expected goals, as an advanced analytic, is typically based on prior distributions of shots taken at similar locations. Why not try to improve this by using other readily available shot data? This paper looks at how to model xG using public data exposed by the NHL API and which features are most valuable in this effort.

1 Introduction

One of the more crude advanced analytics, expected goals (xG) models a team or player’s performance on the basis of the shots they take. A positive difference between xG and goals scored generally means an underperformance of some kind. That could be poor shooting performance by the player or team, bad luck, or high-level play by the opposition goaltender. In contrast, a negative difference between xG and goals scored could mean well-performing shooters, good luck, or poor play by the opponent’s goalie.

A simplistic xG model may use prior shot distributions over time as a base for the calculation. For example, the xG of a shot from position (x, y) on the ice may use shots from the same or similar positions from the previous N seasons.

On the other hand, one may train a non-linear model on shot data. For example, the shot location, angle, shot speed, and more. This is what is done in this paper.

There are two things to note about the usage of expected goals. First, within a game, xG models performance but only up to a point. As with any statistic, sample size matters, and due to the average number of shots per game being roughly 27 for each team, xG becomes inaccurate. Over a season is when it is obvious whether a player or team may have outperformed or underperformed their xG. Second, expected goals does not typically take into account the shooter or goaltender. As an objective statistic, using shooter or goaltender in the model would defeat the purpose. This has been a point of contention amongst hockey fans, however the point of the statistic to measure the quality of the chance. Factoring in the quality of the player or goalie would only serve as noise. Thus, while a sharp angle shot by Leon Draisaitl should warrant more xG than the same shot by Cody Ceci, they are both taking the same poor shot.

2 Data

All data was collected using the NHL’s public API. Unique IDs for games, including preseason, regular season, and playoffs, were pulled from each team’s season schedule. Then, the play-by-play data from those games was accessed using the unique IDs. Finally, events were aggregated such that only shots were entered into the dataset. This led to about 85k shots per year. This model was trained on the past 3 seasons: 2022-2023, 2023-2024, and 2024-2025.

3 Feature Engineering

A number of features were used, and then statistical tests were performed to decide whether to prune certain ones. First, the x and y coordinates were added to the feature set, as well as the distance from the net and the angle to the net, which were computed using x and y .

$$\text{distance} = \sqrt{(89 - x)^2 + (0 - y)^2} \quad (1)$$

The distance formula used was the euclidean distance from the net to the location of the shot. If the rink were on a 2D grid, the net would be at position $(89, 0)$, which is why those constants appear in equation (1).

Next, the angle was computed using the element-wise arc tangent of $\frac{|y|}{89 - x}$, while taking into account the quadrant of the angle. What this means is that, since we have the net at one position $(89, 0)$, it is in the top-left quadrant. Thus, we want our angle to be there and not on the right side of the 2D grid, as it normally would be.

$$\arctan 2(|y|, 89 - x) \quad (2)$$

Note that in equation (2), $\arctan 2$ is the numpy function. There is no doubling of the values going into the arc tangent, nor is there a doubling of the result.

Some features taken into account were the shot type (e.g: wrist shot, slap shot, tip-in, ...), period of play, previous event type (e.g: shot on goal, faceoff, missed shot, ...), and time/distance since previous event.

The previous event type was pulled from the API as the event immediately preceding a goal or shot on goal. The time between these events was determined by the difference in period time remaining between the occurrences

of the events. Thus, a goal scored at 12 : 13 remaining in the period after a faceoff at 12 : 15 would be a two-second difference. The distance from the previous event uses the euclidean distance between the positions of the current shot or goal and the previous event.

The last features used were the change in angle between the last and current event, whether the chance was on the rush, and whether the chance was a rebound.

The angle between events is meant to act as an east-west movement feature. Since the NHL API does not track passes, there was no way of truly determining whether cross-ice or cross-crease passes were made. It seems intuitive that a cross-crease tap in should have a higher xG than a tap-in from a pass only a few feet away. The goalie needing to cross more ground would imply a more open net. To get the angle difference, we computed the absolute difference between the angles of the previous and current event. These angles were calculated using equation (2).

Because the public API does not expose player locations on events other than the player related to the event, there was no way to accurately tell whether a chance was a rush of some kind. Rush chances would probably merit a higher xG than non-rush chances. As an attempt to model this, the distance in the x direction between the previous and current event was used. Recall this has been precomputed, as explained above. If the difference between the previous and current event's x position was greater than 25, it was considered a rush chance. This is an extremely crude heuristic and without player location data or pass data, it cannot be made robust.

Whether the chance was a rebound or not was calculated using the time between the previous shot on goal and the current shot. The heuristic used was as so: if two shots on goal occurred within 2 seconds of each other, with nothing in between, it was likely a rebound. Unfortunately, millisecond-level data was not available, so the data had to be precise to the second instead.

4 Model

This xG model was trained using sklearn's HistGradientBoostingClassifier, a more advanced implementation of the Gradient Boosting algorithm. As seen in the name, this model uses histogram-based techniques to perform the gradient boosting algorithm.

The idea behind the base model is to build trees sequentially which correct the errors of the previous trees. The histogram model also does this, however it splits continuous features into bins (hence the histograms) as opposed to looking at every unique feature value. This has the benefit of speeding up training, as there are fewer bins than there are unique values of a continuous feature in a large dataset. It also has built-in NaN support, by putting these values in their own bin. The model will then decide based on the gradient whether to put this in the left or

right child of the tree.

The purpose of a decision tree for this model is simply for ease of use. Since it is not linear, the result is a more expressive model while not requiring much data preprocessing at all.

5 Results

First, some statistical tests were done to see whether the model was performing well given the features. As mentioned above, some features used other features in their calculation, so there may have been redundancy which these checks aim to expose.

First, a drop-feature test was performed a number of times until no features were worth dropping. The results of the test using all features are shown in figure 1.

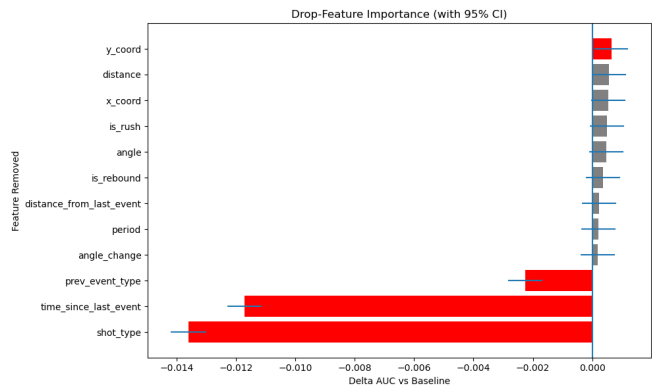


Figure 1: Drop-feature test with no features dropped. Red indicates statistically significant.

Immediately, notice that `y.coord` is actively hurting the model's performance. The receiver operating characteristic area under the curve is lower with that feature than without it. Thus, this is an easy one to remove and the testing continues without it.

This metric, receiver operating characteristic (ROC) area under the curve (AUC), is a performance metric for binary classification models. It measures a model's capability of distinguishing between the two classes across all probability thresholds. If ROC AUC were 1, there would be a 100% probability that the model would correctly classify a random positive sample higher than a random negative sample.

In figure 2, there are no statistically significant values that hurt the model, however `x.coord` is still a viable candidate. Thus, the test was run again without this feature.

The reason for the removal of `x.coord` without any fuss is that the position of the shot (x, y) is taken into account in both distance and angle. There is no point in keeping the redundant x and y values in that case, so their removal becomes quite easy. Not only does the model improve in terms of AUC, but it gets simpler.

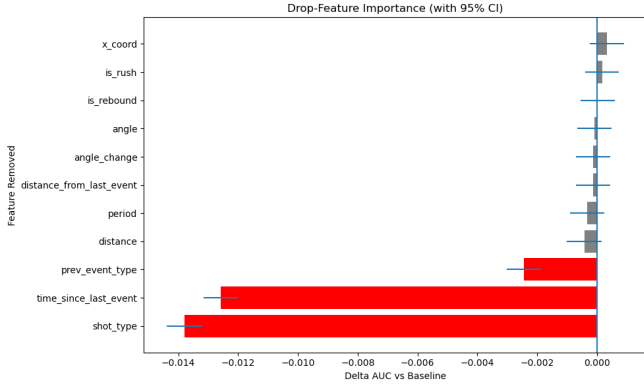


Figure 2: Drop-feature test with `x.coord` dropped. Red indicates statistically significant.

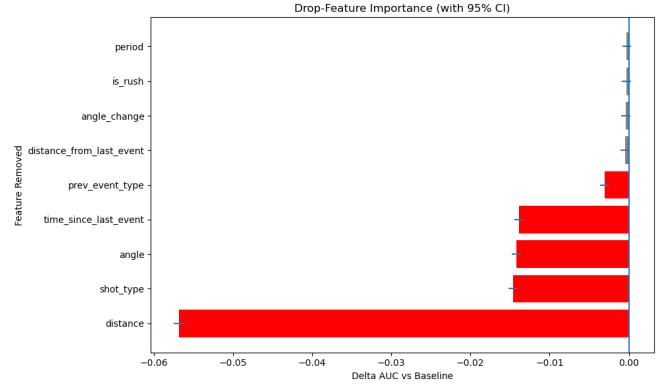


Figure 4: Drop-feature test with `x.coord`, `y.coord`, and `is_rebound` dropped. Red indicates statistically significant.

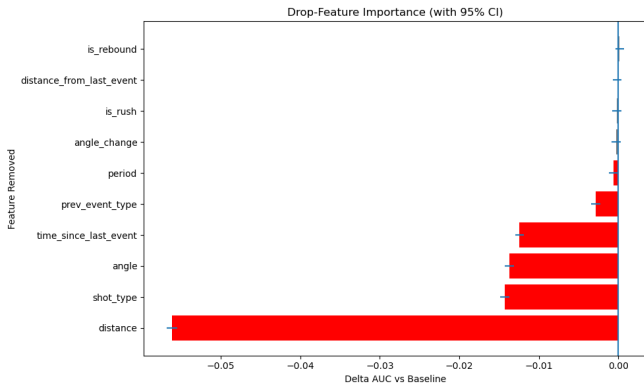


Figure 3: Drop-feature test with `x.coord` and `y.coord` dropped. Red indicates statistically significant.

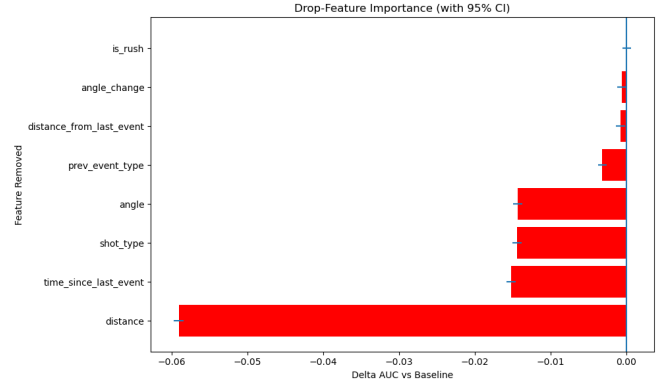


Figure 5: Drop-feature test with `x.coord`, `y.coord`, `is_rebound`, and `period` dropped. Red indicates statistically significant.

After removing `x.coord`, the test is performed again. Figure 3 shows that there are no four features that are neither helping nor hindering the model. Removing any one of these may affect the test results for all the other features. Thus, the next feature to be removed was `is_rebound`. The options were between this and `is_rush` simply because these two are the least robust. They are inherently flawed due to the lack of data in the public NHL API. After flipping a coin, `is_rebound` was removed from the dataset.

In this iteration, the results of figure 4 show that `period`, `is_rush`, `angle_change`, and `distance_from_last_event` are essentially useless. The first feature to be removed was `period`, followed by `is_rush`.

Notice in figures 5 and 6, that the removal of both features affected the model more than just removing `period`. Specifically, removing both made it such that `distance_from_last_event` and `angle_change` were not statistically significant. Thus, the final model would keep `is_rush` and only remove `period`. Largely, the reason for this is that the distance and angle change from the previous event seemed moderately important. As mentioned in the feature engineering section, these features act as

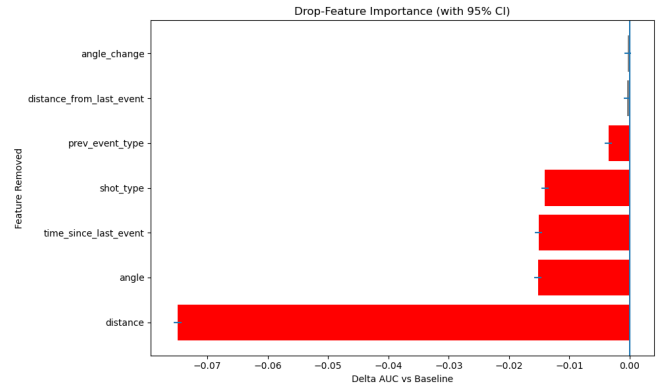


Figure 6: Drop-feature test with `x.coord`, `y.coord`, `is_rebound`, `period`, and `is_rush` dropped. Red indicates statistically significant.

a way to determine how much the goalie had to move across his net. This allows the model to get past the lack of passing data.

Prior to training, mutual information was computed on the remaining features of the training set. Mutual information is a way to determine which features are most informative for predicting the target. In this case, which

features are most informative for deciding whether a shot ends in a goal or not.

Importantly, mutual information does not assume linear relationships between features, so more complex patterns may be captured by the test.

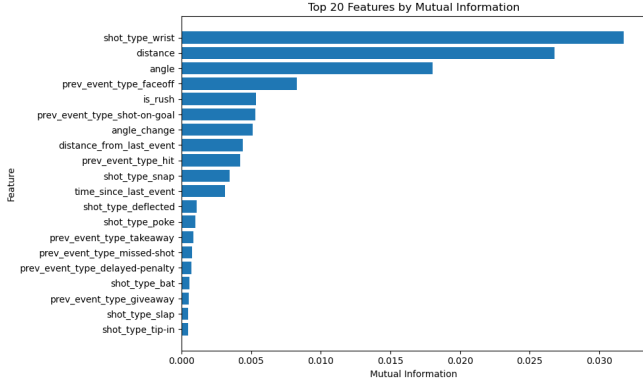


Figure 7: Mutual information of top 20 features in training set.

In figure 7, the most informative features are whether the shot was a wrist shot, the distance from the net, and the angle at which the shot was taken. Distance and angle are unsurprising, however wrist shots being most informative is quite interesting.

The next most informative feature was the previous event being a faceoff. There are two options here: either faceoff set plays lead to goals quite often or shots after the faceoff generally do not go in all that much.

After that, a number of features are mildly informative, but not impressive enough to mention. Of the least informative bunch, deflected shots are very surprising. It's quite well established that the puck changing direction at the last second is the hardest thing for a goalie to stop, so seeing deflections be so uninformative for the model is curious. Most likely, deflected shots end up being too close to the net and thus not changing direction enough and the goalie just happens to be in the right position. Therefore, deflected shots do not go in as often as expected.

The model was trained and tested in the same way for all the drop feature tests and for its final run. That is, a 3-fold cross-validation and hyperparameter search on learning rate, maximum depth, and maximum iterations. Overall, each test required 24 runs of the training process. These were parallelized on CPU and thus took about 5 seconds each. The entire training process, including all cross-validation and tuning, took roughly 15 seconds. The entire dataset was around 250k samples, and the training set was 80% of that.

The best cross validation area under the curve (AUC) for the final model was 0.764. This is better than the previous best over all prior models (the ones with fewer features removed) by about 0.1. While this is not much, it does matter since AUC is between 0 and 1.

Before testing the final model, importance was com-

puted and plotted. Importance essentially shows how much the model used a specific feature in inferring the target. It is different from mutual information in that it depends on the model. Mutual information is a mathematical formula based on the training data alone.

Specifically, this model used SHAP importance. Compared to base importance, SHAP importance has a direction and magnitude. We can essentially describe this as a vector while base importance is a scalar value. A more complete explanation will follow using the figure.

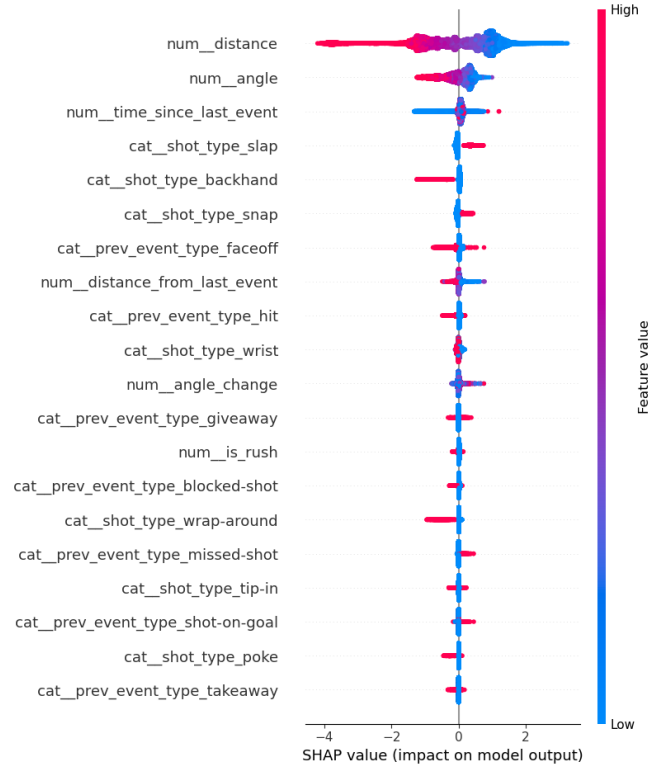


Figure 8: SHAP importance of top 20 features in dataset.

In figure 8, notice that SHAP value is the impact on the model output. What this means is that a negative SHAP value corresponds to the model predicting a lower xG. Then, notice that a red feature value is high, and a blue value is low. Thus, when the distance from the net is high, the model predicts a lower xG, and when a shot is taken from in close, the model predicts a higher xG. This is exactly as expected from the model.

Similarly, a greater angle makes it such that the model predicts a lower xG. Recall that, since the angle is from the centre of the net, an angle of 0° would be straight-on, while an angle of 90° would be a poor shot from the side of the net.

Some other interesting features are that backhands and wrap-arounds are modeled as lower xG by the model. These are generally poor angle or low percentage shots and rely on specific goalie movement or shot placement to go in. That is, a backhand typically means the shooter is on a breakaway and making a move or is in tight and

can't get to his forehand. The second case seems more common and would lead to poor shots. A wrap-around is modeled as lower xG by the model most likely because goalie position data is unavailable. These shots work best when the goalie is out of position. Thus, it would be interesting to look at the interaction between the prior event being a missed shot and the current shot being a wrap-around. If a shot is missed behind the net, the goalie may be out of position, allowing a player to score on a wrap-around.

Finally, after all this, the model was tested on the test set of about 25k samples. The final ROC AUC was 0.768, quite good on the unseen set of samples. As mentioned above, ROC AUC has a limit of 1 in the best case. However, for noisy data like this, a perfect shot may not go in due to a super-human effort by the goaltender or a catastrophic miss by the shooter. Think of the 2007 empty net miss by Patrik Stefan: the net was wide open, no one was around him, the puck was flat. Yet he still missed. In the other case, Antti Niemi's sprawling save on Jeff Carter in the 2010 Stanley Cup Finals. The goalie was out of position and flat on his stomach and somehow made a save on Jeff Carter who had the entire net to shoot into.

All this to say that an ROC AUC of 1 is impossible. The result for this model is quite good, and considering the lack of positional data, it is extremely impressive.

6 Conclusion

This project served as both a foray into analytics and a realization that xG is perhaps best left as a simple stat. Obviously, there are ways to improve the model, but the results make it clear that in a vacuum, shot location reigns supreme.

This model's greatest strength is its simplicity. It does not access player or goalie skill level and uses basic shot location and timing data to compute higher-order features. The addition of categorical variables such as `previous_event_type` and `shot_type` allowed the model to predict goals with better confidence.

On the other hand, the model was limited by lack of access to certain features. Position data for other players on the ice, in particular the goalie, would have been most useful in improving the model. Even without giving the model access to that raw data, using it to determine whether a player was on a breakaway or two-on-one would increase ROC AUC significantly.

In the future, comparing this model to one with only distance and angle as features would be an interesting test. Seeing as those two features led in SHAP importance and were in the top three of mutual information, it seems obvious to conclude that they are the most valuable features. In the same vein, perhaps adding features would make for an interesting comparison. Specifically, adding shooter and goalie identification values.

Shooters take around 100 shots per season on average.

This is a very rough estimate and does not take into account ice time, position, or whether the player has also played in the playoffs. In any case, over a number of seasons, the model should have enough data to confidently conclude what a shot's xG would be given the player taking the shot and against whom they are shooting.